Constants

While variables are designed to change, sometimes you need data to stay permanently locked in place while your application runs. Go handles this using **Constants** and a special keyword called **iota**.

1. Constants (const)

A constant is a value that is strictly locked in at **compile time**. Once you set it, the program is physically incapable of changing it while running. You use the **const** keyword instead of **var** or **:=.**

- **Grouping:** You can declare multiple constants at once to keep your code clean:

```
const (
   y = 4
   z = "hi"
)
```

2. The iota Keyword (Go's Version of Enums)

In traditional **OOP** languages (like **C++**), you have **"Enumerated Types"** (Enums). These are used when you have a specific list of related states, like the days of the week, or grades (A, B, C, D, F).

Go doesn't have an **enum** keyword. Instead, it uses **iota**.

- **The Goal:** You want the computer to know that "Monday" is distinctly different from "Tuesday," but you don't actually care what raw number the computer assigns to them behind the scenes.
- **How it works:** When you create a grouped list of constants, you set the very first one to iota. Go will automatically assign it a starting number, and then magically auto-increment the value for every subsequent constant in the list without you typing anything else!

```
type Grade int
const (
   A Grade = iota  // Go assigns this automatically
   B               // Go knows this is a Grade and increments the iota value
   C               // Increments again...
   D
)
```

Expert Takeaway:

When building robust web applications or setting up a database schema, iota is a massive performance hack. Instead of saving user states in your database as heavy text strings (like **"PENDING", "ACTIVE", or "SUSPENDED"),** you map them to an iota list in Go.

Your backend logic remains incredibly readable **(e.g., if user.Status == ACTIVE),** but under the hood, Go and your database are just passing around tiny, lightning-fast integers like 0, 1, and 2.